

Apuntes de
Diseño y Análisis de Algoritmos
Tomados como estudiante y revisados como
profesor

Federico Melo Barrero
f.melo@uniandes.edu.co

Copyright © 2022, 2026 Federico Melo Barrero.

Todos los derechos reservados.

Ninguna parte de este documento, incluyendo el diseño de la portada, figuras e imágenes, puede ser reproducida, registrada, almacenada o transmitida en manera alguna ni por ningún medio, ya sea electrónico, eléctrico, fotoquímico, mecánico, electroóptico, magnético, de grabación, de fotocopia o cualquier otro sin permiso previo y por escrito del autor.

Índice

I	Introducción	6
1.	Algoritmo	6
2.	Seudocódigo	6
3.	Problema	7
3.1.	Tipos de problemas	8
II	Diseño de algoritmos	9
3.2.	Insertion sort	9
4.	Recursión	10
4.1.	Sucesión de Fibonacci	10
4.2.	Ejecución de la recursión	10
4.2.1.	Árbol de ejecución	10
4.2.2.	Orden de ejecución	11
4.2.3.	Pila de ejecución	12
4.3.	Complejidad de la recursión	12
4.3.1.	Complejidad temporal	12
4.4.	Equivalencia entre recursión e iteración	12
5.	Dividir y conquistar	13
5.1.	Búsqueda binaria	13
5.1.1.	Solución recursiva	13
5.1.2.	Solución iterativa	14
5.2.	Merge sort	15
5.2.1.	Dividir y conquistar el arreglo	15
5.2.2.	Combinar las mitades	16
5.3.	Subarreglo máximo	17
5.3.1.	Solución por fuerza bruta	17
5.3.2.	Solución por dividir y conquistar	18
6.	Programación dinámica	19
6.0.1.	Cuándo usar programación dinámica	19
6.0.2.	Tipos de programación dinámica	19
6.0.3.	Recomendaciones	20
6.1.	Sucesión de Fibonacci	21
6.1.1.	Subproblemas superpuestos	21
6.1.2.	Solución top-down	21
6.1.3.	Solución bottom-up	22
6.2.	Problema de corte de vara	22
6.2.1.	Solución recursiva	23
6.2.2.	Solución top-down	23
6.2.3.	Solución bottom-up	24
6.2.4.	Obtener los tamaños de los cortes	24
6.3.	Multiplicación de matrices en cadena	25
6.3.1.	Costo computacional	25
6.3.2.	Subestructura óptima	26
6.3.3.	Formulación recursiva	27

7. Algoritmos aproximados	27
7.1. Razón de aproximación	27
7.2. Problema de vertex-cover	28
7.2.1. Algoritmo aproximado para vertex-cover	28
7.2.2. Complejidad temporal	28
7.2.3. Razón de aproximación	28
 III Análisis de algoritmos	 29
8. Correctitud	29
9. Eficiencia	29
9.1. Complejidad temporal	29
9.1.1. Modelo de computación RAM	30
9.2. Análisis exacto	30
10. Intratabilidad	31
10.1. P vs NP	32
10.2. Problemas de decisión	32
 IV Apéndices	 33
Apéndice A. Notación	33
Apéndice B. Inducción	33
Apéndice C. Ecuaciones recursivas	33
3.1. Relación con ecuaciones diferenciales	33
Apéndice D. Multiplicación matricial	33
Apéndice E. Números de Catalán	34

Prefacio

Tomé estos apuntes cuando cursé Diseño y Análisis de Algoritmos en la Universidad de los Andes, entre agosto y noviembre de 2022.

Posteriormente los fui corrigiendo y extendiendo intermitentemente a medida que trabajaba en temas relacionados con algoritmia. Las extensiones más notables corresponden a dos momentos: cuando cursé Teoría de Grafos entre agosto y noviembre de 2023 y cuando fui seleccionado para dictar el curso de Diseño de Algoritmos en mayo de 2026.

Los apuntes están presentados en el mismo formato en el que originalmente fueron escritos: la clase de $\text{\LaTeX}$ que establece su estilo la escribí en noviembre del 2020. Más importante aún, los apuntes preservan la estructura y el tono de cuando los escribí como estudiante, con la intención de que sean útiles precisamente a esa audiencia.

La selección deliberada de la audiencia es en gran medida lo que otorga valor a estas notas. Textos clásicos como CLRS¹ intentan dirigirse a una audiencia mixta: estudiantes, profesores, investigadores y profesionales. Esa amplitud de audiencia genera que el texto se difunda ampliamente, pero también implica un nivel de formalidad, profundidad y exhaustividad que muchas veces no es el más adecuado para un primer encuentro con el tema, especialmente a nivel de pregrado.

Estas notas no sacrifican rigor ni correctitud, pero sí priorizan claridad pedagógica sobre exhaustividad formal. En consecuencia, omiten algunos detalles cuya inclusión sería valiosa en un texto de referencia, pero que para muchos estudiantes solo alargarían el texto y añadirían complejidad innecesaria.

¹Uso esta sigla para referirme al famosísimo Introduction to Algorithms de Cormen, Leiserson, Rivest y Stein, que es un excelente ejemplo de este fenómeno: con docenas de miles de citas en google scholar, claramente tiene la profundidad suficiente para resultar útil a investigadores.

Prerrequisitos

Estos apuntes están escritos de forma que los prerrequisitos para entenderlos son sorprendentemente laxos para el nivel de material que se cubre. Hay que saber lo siguiente:

- Programación básica en algún lenguaje de programación.
 - El pseudocódigo que se emplea es bastante parecido a Python en la sintaxis; sin embargo, es legible si se tiene experiencia con cualquier lenguaje de programación imperativo.
- Conocimiento básico de las estructuras de datos abstractas fundamentales: arreglos, listas, pilas, colas, árboles y tablas de hash; qué operaciones abstractas tiene cada una y su costo computacional.
- Matemática básica
 - Notación matemática básica: conjuntos, operadores, cuantificadores, etc.
 - Aritmética básica: sumar, multiplicar, potenciar, etc.
 - Álgebra elemental: despejar variables, factorizar, etc.
 - Funciones elementales: logaritmos, exponenciales, polinomios, etc.

Parte I

Introducción

1. Algoritmo

Estas notas estudian algoritmos. Un **algoritmo** es una secuencia de pasos. Pero nos interesan los algoritmos que resuelven un problema de computación, entonces:

Definición 1.1. Algoritmo

Secuencia de pasos para transformar un conjunto de valores de **entrada** (*input*) a un conjunto de valores de **salida** (*output*).

No se escribe “a otro conjunto de valores” porque la salida podría ser igual a la entrada.

Un algoritmo se puede expresar en:

- Lenguaje natural (español, inglés, etc.)
- Lenguaje formal (matemática o algún lenguaje de programación o lógico)
- Seudocódigo (una mezcla de los dos anteriores)
- Como programa de computador
- Como hardware (esto es difícil de imaginar en este punto, pero sí)

2. Seudocódigo

TL;DR Se utiliza unseudocódigo inspirado en Python, excepto cuando una notación más universal, matemática o algorítmicamente explícita permite expresar mejor la idea.

Elseudocódigo es una forma de describir algoritmos parecida a los lenguajes de programación, pero que permite despreocuparse de detalles de sintaxis o de implementación, que no suelen ser relevantes al estudiar algorítmica, para enfocarse en resaltar las ideas del algoritmo.

No existe una única forma de escribirseudocódigo. Distintos libros, cursos y autores utilizan convenciones diferentes inspiradas en distintos lenguajes de programación. En los textos formales se definen convenciones estrictas para elseudocódigo que se usa; eso elimina la ambigüedad, pero obliga al lector a aprenderse las convenciones, prácticamente un nuevo lenguaje.

En estos apuntes se utiliza unseudocódigo inspirado en Python, que es un lenguaje con una sintaxis breve y legible. Como el objetivo no es aprender Python sino estudiar algoritmos, ocasionalmente se reemplazan construcciones específicas del lenguaje por notaciones más universales o explícitas. En particular, cuando una sintaxis oculta detalles relevantes para el análisis algorítmico, se prefiere una descripción que haga visibles dichos detalles. No usar trucos específicos de Python permite además que cualquier lector que conozca algún otro lenguaje imperativo pueda leer el código sin problema.

E.g., una función que recibe un arreglo de números y encuentra el mayor:

```

1 def find_largest(nums: arreglo[int]) -> int:
2     largest = -∞
3     for i=0 to len(nums)-1:
4         if nums[i] > largest:
5             largest = nums[i]
6     return largest

```

En ese ejemplo, es evidente que el código se asemeja a Python, salvo algunos detalles. Los detalles más recurrentes en estas notas donde el pseudocódigo difiere con Python son los siguientes:

- Al utilizar cualquier estructura de datos abstracta, como arreglos, colas, pilas, árboles, etc, la estructura se declara en español.
 - Por ejemplo, se utiliza el tipo `arreglo` en lugar de utilizar objetos de la clase `list`, para que tenga las propiedades de un arreglo abstracto, como tamaño fijo y acceso por índice en tiempo constante, en lugar de las del tipo `list` que en Python corresponde a un arreglo dinámico.
- Los ciclos sobre rangos de enteros se escriben como `for i=0 to n-1` en lugar de utilizar `for i in range(n)`. La notación preferida es familiar para quienes provienen de otros lenguajes imperativos y además hace explícitos los límites del recorrido.
- En lugar de usar *slicing*, se escribe literalmente (en español) que se realiza una copia y de qué segmento específico del arreglo.
- Cuando se usa la noción de infinito, se escribe el símbolo matemático ∞ en lugar de la sintaxis `float('inf')`.
- Se evita usar cualquier función, como `sorted`, o librería, como `itertools`, que encapsule operaciones complejas, pues el propósito es precisamente comprender cómo se implementarían desde cero.

Con lo anterior en mente, debería ser trivial traducir cualquiera de los algoritmos en estas notas a Python ejecutable, y también debería ser sencillo traducirlos a cualquier lenguaje imperativo.

3. Problema

Un problema (en el contexto de estas notas) es un enunciado que especifica qué se quiere obtener, sin decir cómo. Más formalmente:

Definición 3.1. Problema (computacional)

Un **problema** es una especificación de una entrada y de una salida esperada con base en la entrada.

E.g., el **problema de ordenamiento** (*sorting problem*, muy famoso):

- **Entrada:** secuencia de n números $a_1, a_2, \dots, a_n$.
- **Salida:** permutación $a'_1, a'_2, \dots, a'_n$ tal que $a'_1 \leq a'_2 \leq \dots \leq a'_n$

Una **permutación** de una secuencia es un posible orden de sus elementos; es como decir, coloquialmente, un orden. Si la secuencia tiene más de un elemento, tiene muchas permutaciones. En este caso, el problema especifica como salida esperada una permutación en la que los números están ordenados de menor a mayor.

Usualmente los problemas están escritos de forma que la identificación de la entrada y la salida esperada no es tan inmediata. Como el objetivo de estas notas es el diseño de algoritmos y no la comprensión lectora, siempre se especifican los problemas enunciando directamente la entrada y salida esperadas.

Una **instancia** de un problema es una entrada específica, e.g., $1, 4, 2, 3, 2$ es una instancia del problema de ordenamiento, en cuyo caso una **solución** es $1, 2, 2, 3, 4$.

3.1. Tipos de problemas

Hay muchos tipos de problemas computacionales. En general, un mismo problema de la vida real se puede modelar de varias formas, que pueden corresponder a un tipo de problema computacional diferente. Algunos tipos son:

- **Problemas de enumeración:** la salida son todas las soluciones válidas.
- **Problemas de conteo:** la salida es el número de soluciones válidas.
- **Problemas de optimización:** la salida es la mejor solución válida de acuerdo a algún criterio.
- **Problemas de búsqueda:** la salida es alguna solución válida para el problema, si existe.
- **Problemas de decisión:** la salida es un sí o un no.

Estas categorías no son mutuamente excluyentes, muchas veces son distintas perspectivas del mismo problema. Por ejemplo, para el problema de ordenamiento se puede abordar bajo cualquiera de las perspectivas anteriores:

- Enumeración: dado un arreglo, listar todas las permutaciones ordenadas de sus elementos.
- Conteo: dado un arreglo, determinar cuántas permutaciones distintas de sus elementos están ordenadas.
- Optimización: dado un arreglo y un conjunto de operaciones permitidas, encontrar una secuencia de operaciones que lo ordene usando el menor número de operaciones posible.
- Búsqueda: dado un arreglo, encontrar alguna permutación de sus elementos que esté ordenada.
- Decisión: dado un arreglo, ¿está ordenado?

Se pueden formular aún más problemas relacionados a ordenar elementos que encajen en cualquiera de estas categorías. En este punto, no es tan inmediato determinar cómo traducir entre tipos de problemas, en gran parte porque las definiciones anteriores no son matemáticamente formales, sino más bien la intuición detrás de cada tipo de problema. Por lo pronto, al diseñar algoritmos nos ocuparemos de indicar qué tipo de problemas se pueden abordar con la técnica que se está presentando, sin perder de vista que muchos problemas admiten reformulaciones que los hacen encajar en distintos tipos.

Parte II

Diseño de algoritmos

En esta parte de las notas se presentan distintas técnicas para diseñar algoritmos. No se presentan en orden de dificultad, principalmente debido a que la dificultad es subjetiva: cada método es difícil por una razón distinta. Pero sí se presentan en un orden conveniente, para que al explicar métodos posteriores se puedan dar por sentados conceptos de métodos anteriores.

3.2. Insertion sort

Insertion sort resuelve el problema de ordenamiento, definido en la sección 3. Es quizás el algoritmo de ordenamiento más intuitivo.

Se recorren los elementos del arreglo. Para cada elemento, se revisa si está ubicado correctamente respecto a los anteriores: si no lo está, se mueve hacia atrás hasta que lo esté; una vez lo está, se continúa con el siguiente elemento.

```

1 def insertion_sort(nums: arreglo):
2     i = 1
3
4     while i < len(nums):
5         elem = nums[i]
6         j = i-1
7         while j >= 0 and elem < nums[j]:
8             nums[j+1] = nums[j]
9             j -= 1
10
11         nums[j+1] = elem
12         i += 1

```

Se inicia revisando el segundo elemento, para poder compararlo con algún anterior.

Si `elem` es menor que el anterior y debe moverse hacia atrás, se mueve el anterior una posición adelante para hacerle espacio. Se siguen corriendo los anteriores hasta que ya no deba moverse `elem` hacia atrás (o ya no hayan anteriores).

Se inserta `elem` en la posición que quedó libre tras mover los elementos más grandes que él hacia adelante. Es `j+1` y no `j` porque la última iteración del ciclo termina con `j -= 1`.

El algoritmo presenta un ciclo anidado dentro de otro. El ciclo externo siempre recorre el arreglo completo, pero la cantidad de veces que ejecuta el cuerpo del ciclo interno depende de qué tanto haya que retroceder cada elemento, es decir, de qué tan ordenado esté el arreglo.

El mejor caso es cuando el arreglo está ordenado, pues no hay que retroceder ningún elemento y el cuerpo del ciclo interno nunca ejecuta. En ese caso el costo computacional está dado solo por el ciclo externo, que indefectiblemente revisa todos los elementos. Sea T_{mejor} la función para ese costo computacional, $T_{\text{mejor}} \in \Theta(n)$.

El peor caso es cuando el ciclo interno tiene que retroceder cada elemento hasta el principio, es decir, cuando el arreglo está en orden inverso. En ese caso, para cada elemento i , el ciclo interno debe retrocederlo $i - 1$ veces; la cantidad de operaciones es entonces:

$$\sum_{i=2}^n (i-1) = 1 + 2 + \dots + (n-1) = \frac{n^2 - n}{2}.$$

Eso, aunado a las operaciones del ciclo externo, da la siguiente función para el costo computacional, donde c denota el tiempo que demoran las operaciones constantes:

$$T_{\text{peor}}(n) = cn + c \frac{n^2 - n}{2}$$

Ergo, $T_{\text{peor}} \in \Theta(n^2)$. Convencionalmente, se dice que insertion sort es $\Theta(n^2)$, entendiendo que se hace referencia específicamente al peor caso.



Tip. Para resolver la sumatoria se empleó la fórmula de Gauss, cuyo argumento subyacente es bastante simple y se expone en el Apéndice A.

4. Recursión

Una función (matemática o de programación) es **recursiva** si está definida en términos de sí misma.

En términos conceptuales, para resolver un problema utilizando recursión, el problema se debe poder dividir en subproblemas más pequeños con la misma estructura del original. Las soluciones a esos subproblemas se utilizan para solucionar el problema original.

En términos de implementación, una función recursiva se debe invocar a sí misma. Por ende, dada una función F , la entrada X de la invocación original $F(X)$ debe poderse dividir en pedazos con su misma estructura (mismo tipo de dato), $x_1 \in X, x_2 \in X, \dots, x_n \in X$, que son usados como argumentos de los llamados recursivos: $F(x_1), F(x_2), \dots, F(x_n)$. Cada uno de esos llamados representa la solución a un subproblema.

Toda función recursiva consta de dos elementos:

- Uno o más casos en los que la función retorna sin invocarse a sí misma, denominados **casos base**.
 - Un caso base se alcanza cuando el subproblema es tan pequeño que puede solucionarse sin recursión.
- Una **relación de recurrencia**, que define la relación entre la solución del problema y la solución de sus subproblemas.
 - Esa relación de recurrencia combina de alguna manera los resultados de los llamados recursivos $F(x_1), F(x_2), \dots, F(x_n)$ para obtener la solución al problema original.

4.1. Sucesión de Fibonacci

Se presenta la sucesión de Fibonacci como ejemplo clásico de recursión. La sucesión comienza con 0 y 1, y cada número posterior es la suma de los dos anteriores. Formalmente:

$$\begin{aligned} F(0) &= 0 \\ F(1) &= 1 \\ F(n) &= F(n-1) + F(n-2) \quad \text{para } n > 1 \end{aligned}$$

Allí, los casos base son las primeras dos expresiones y la relación de recurrencia es la tercera expresión. Se puede traducir esa definición a una implementación recursiva directamente:

```

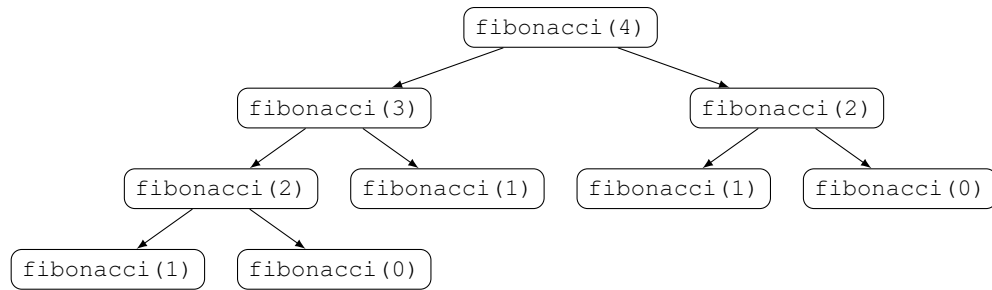
1 def fibonacci(n: int) -> int:
2     if n <= 1:
3         return n
4
5     return fibonacci(n-1) + fibonacci(n-2)
```

4.2. Ejecución de la recursión

Aprovechamos la sucesión de Fibonacci para estudiar cómo ejecuta una función recursiva.

4.2.1. Árbol de ejecución

Cada vez que se invoca `fibonacci`, durante su ejecución se invoca nuevamente `fibonacci` 0 o 2 veces. La manera convencional de representar ese mecanismo es mediante un **árbol de ejecución** en el que cada nodo representa un llamado a la función y sus hijos son los llamados recursivos que hace. Para `fibonacci(4)`:



Nótese que, para toda función recursiva, el árbol de ejecución tiene las siguientes propiedades:

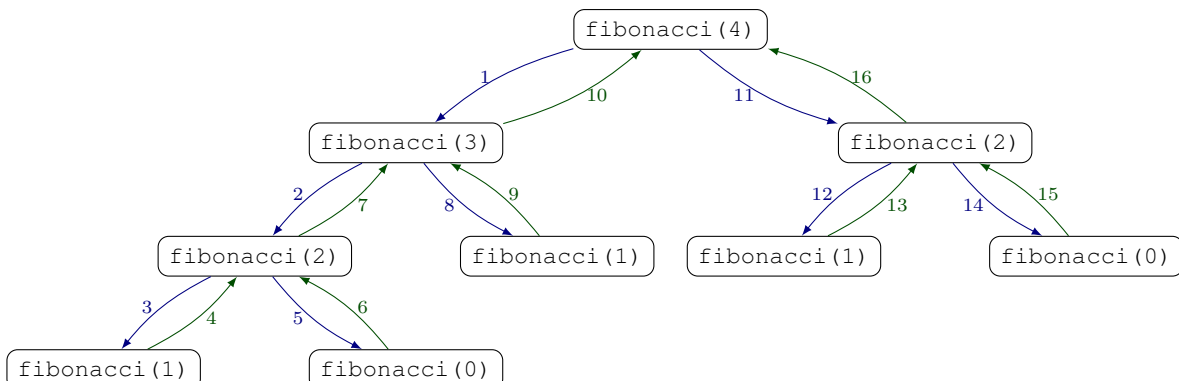
- La raíz del árbol representa la invocación inicial de la función. Por ende, el árbol completo representa la solución del problema original.
- Cada nodo representa una invocación de la función, por lo que el conteo de nodos equivale al total de llamados.
- Cada subárbol representa entonces la solución de uno de los subproblemas y la cantidad de subárboles el número de subproblemas.
- Los hijos representan las llamadas recursivas realizadas por esa invocación.
 - En el caso de `fibonacci`, siempre se realizan 0 o 2 llamados, por lo cual cada nodo tiene 0 o 2 hijos. Eso resulta en un árbol binario.
- Los hojas representan entonces llamados de la función en los que no se realizan llamados recursivos, es decir, en los que se alcanza el caso base.

4.2.2. Orden de ejecución

Al leer un programa iterativo, el orden de ejecución coincide con el orden textual del código: de arriba hacia abajo, repetidamente. Esto no sucede en la recursión, por lo que vale la pena ahondar en cómo entender el orden de los llamados.

Supóngase que se invoca `fibonacci(4)`. El código salta el caso base y llega directamente a la línea de retorno, pero antes de retornar ejecuta el lado izquierdo de la suma, `fibonacci(3)`. Inmediatamente entra en el contexto de ese llamado, antes de ejecutar el lado derecho de la suma. Por ende, todos los llamados recursivos dentro de `fibonacci(3)` ejecutan antes que el lado derecho de la suma; luego todos los llamados recursivos del lado derecho de la suma ejecutan; y, de últimas, en efecto ocurre la suma y termina la ejecución de `fibonacci(4)`.

El árbol de ejecución no enfatiza en el orden en el que se realizan los llamados, por lo que el siguiente grafo dirigido añade aristas numeradas que ilustran el orden de ejecución. Como los hijos son invocados por los padres, las aristas dirigidas hacia abajo denotan que una función invoca a otra; se ilustran en azul. Las aristas dirigidas hacia arriba denotan que la función invocada retorna a la que la invocó y se ilustran en verde.



Como al invocar una función esta ejecuta de forma inmediata, la ejecución recursiva siempre explora completamente una rama del árbol antes de pasar a la siguiente. Con eso en mente y observando el flujo de ejecución expuesto arriba, se puede ver que el orden de ejecución de toda función recursiva está dado por un recorrido DFS sobre el árbol de ejecución.

4.2.3. Pila de ejecución

La inmensa mayoría de las arquitecturas y entornos de ejecución modernos utilizan una **pila de llamados** (*call stack*) para administrar las llamadas a funciones. Se aprovecha la estructura LIFO (*Last-In, First-Out*; última que entra, primera que sale) de la pila para que la última función en ser invocada termine primero.

Cuando una función *A* llama a una función *B*, se crea un **marco de pila** (*stack frame*) con la información de *B*: sus parámetros, las variables locales y la dirección de retorno (la ubicación a la que debe volver la ejecución cuando *B* termine). Al colocar el marco de pila de *B* en la pila de llamadas, queda encima del marco de pila de *A*, que a su vez está encima del marco de pila de la función que la invocó. Así pues, la pila refleja en todo momento la cadena de llamadas activas. Cuando *B* retorna, su marco se elimina de la pila y la ejecución continúa en *A*, donde indica la dirección de retorno.

Si una función se invoca recursivamente, cada llamada recursiva genera un nuevo marco de pila, a pesar de que cada llamado corresponda a la misma función y al mismo código fuente, pues las ejecuciones deben ser independientes. Los llamados se apilan en el orden en el que ocurren, es decir, de acuerdo con un recorrido DFS sobre el árbol de ejecución. A medida que terminan, se remueven de la pila. Por ende, en cada nodo del árbol, la profundidad de la pila (i.e., el número de marcos de pila que tiene) es igual a la distancia de ese nodo a la raíz. Una consecuencia de eso es que la máxima profundidad que alcanza la pila coincide con la altura del árbol de ejecución, que es la longitud de su camino más largo de raíz a hoja. Esa última observación es relevante para el cálculo de la complejidad espacial.

4.3. Complejidad de la recursión

Al igual que el orden de ejecución de una función recursiva no es evidente a primera vista, la complejidad tampoco suele serlo.

4.3.1. Complejidad temporal

4.4. Equivalencia entre recursión e iteración

A pesar de que algunos problemas se expresan más naturalmente mediante recursión que mediante iteración, para todo algoritmo recursivo existe un algoritmo iterativo equivalente.

Para comprobarlo, primero nótese que todos los tipos de iteración se pueden generalizar mediante la instrucción ‘while’, como se muestra en el Apéndice B. Si *B* es una metavariante que representa cualquier expresión booleana y *S* otra que representa cualquier sentencia o instrucción, la forma general de cualquier ‘while’ (y por ende de cualquier ciclo) es:

```
1 while B:
2     S
```

A esa abstracción de los programas iterativos, le corresponde el siguiente programa recursivo equivalente:

```
1 loop():
2     if not B:
3         return
4     S
5     loop()
```

En ambos programas se realiza *S* mientras que *B* sea verdadero y se deja de realizar *S* en cuanto *B* se torna falso.

Si $v_1, \dots, v_n$ representan todas las variables cuyo valor puede influir en la condición o ser modificado por el cuerpo del ciclo, entonces la abstracción de un ciclo pasa a ser algo como:

```
1 while  $B(v_1, \dots, v_n)$  :
2     actualizar( $v_1, \dots, v_n$ )
```

La equivalencia recursiva es

```
1 loop( $v_1, \dots, v_n$ ) :
2     if not  $B(v_1, \dots, v_n)$  :
3         return (...)
4
5     return loop( $v'_1, \dots, v'_n$ )
```

Donde v'_i son los valores después de ejecutar el cuerpo.

5. Dividir y conquistar

Muchos problemas pueden dividirse en subproblemas más pequeños y esos subproblemas siguen siendo muy similares al problema original. Por ejemplo, organizar su casa se puede dividir en organizar la cocina, la sala, etc. que son problemas parecidos pero con menos cosas para organizar.

Dividir y conquistar es la técnica de diseño de algoritmos que formaliza esa idea. Consiste en:

1. **Dividir** el problema en subproblemas más pequeños (usualmente en la mitad)
2. **Conquistar** (resolver) cada subproblema de forma recursiva (o, si el subproblema es muy pequeño, resolverlo directamente)
3. **Combinar** las soluciones de los subproblemas, obteniendo la solución del problema original. Este último paso no siempre es necesario

Se presentan cuatro algoritmos para concretar esta idea.

5.1. Búsqueda binaria

La búsqueda binaria es un algoritmo para buscar si un elemento específico existe en una secuencia ordenada. El problema que resuelve es entonces:

- Entrada: arreglo no vacío ordenado de enteros y número entero k
- Salida: índice de k en el arreglo o -1 si no está

5.1.1. Solución recursiva

El algoritmo es muy simple: se revisa el elemento del medio del arreglo. Si es k , encontramos lo que buscábamos y la salida es el índice del medio; si es menor, solo es sensato seguir buscando en la mitad derecha; si es mayor, solo en la mitad izquierda.

```

1 def binary_search(nums: arreglo[int], target: int) -> int:
2     return binary_search_rec(nums, target, 0, len(nums) - 1)
3
4 def binary_search_rec(nums: arreglo[int], target: int, i: int, j: int) -> int:
5     if j < i:                } Si j está a la izquierda de i, los índices se cruzaron,
6         return -1          } lo que indica que el elemento no está.
7
8     k = (i + (j - i) // 2) ...> Una alternativa para calcular la mitad es (i+j) // 2, pero arriesga
9                                un overflow (en lenguajes de bajo nivel) si la suma es muy grande.
10    if target < nums[k]:
11        return binary_search_rec(nums, target, i, k - 1)
12    if target > nums[k]:
13        return binary_search_rec(nums, target, k + 1, j)
14
15    return k

```

Nótese que en los llamados recursivos se toma desde el índice de la mitad, k , *más uno* (o menos uno). Eso cumple dos funciones: primero, que no se vuelva a revisar la mitad; pero más importante aún, permite que a medida que el número no se encuentra y el segmento del arreglo que se contempla es cada vez más pequeño, de forma que el índice de la mitad se acerca más y más al extremo, eventualmente se crucen y la recursión llegue al caso base. Si solo se tomara el índice de la mitad, jamás se cruzaría con el extremo y la recursión no terminaría. Y no se puede detener cuando son iguales, porque ese caso de un único elemento sí se quiere revisar.

Como cada vez que se parte el problema se reduce a la mitad, para determinar la complejidad se quiere saber cuántas veces se debe dividir la entrada de tamaño n entre 2 hasta que se llega a un problema de tamaño 1. Para conocer esa x cantidad de veces, se formula:

$$n \cdot \underbrace{\frac{1}{2} \cdot \frac{1}{2} \cdots \frac{1}{2}}_{x \text{ veces}} = 1$$

$$\frac{n}{2^x} = 1$$

Despejando,

$$2^x = n$$

$$x = \log_2(n)$$

Es decir, el número de divisiones, que es el número de pasos, crece logarítmicamente con respecto al tamaño de la entrada.

5.1.2. Solución iterativa

En este algoritmo, solo se realizan las etapas de dividir y conquistar; no es necesario combinar las soluciones. Eso facilita realizar una implementación iterativa de búsqueda binaria:

```

1 def binary_search_iter(nums: arreglo[int], target: int) -> int:
2     i = 0
3     j = len(nums) - 1
4
5     idx = -1
6
7     while i <= j and idx == -1:
8         k = i + (j-i) // 2
9         if target < nums[k]:
10             j = k - 1
11         elif target > nums[k]:
12             i = k + 1
13         else:
14             idx = k
15
16     return idx

```

Práctica: 🐞 Binary Search

5.2. Merge sort

Merge sort resuelve el problema de ordenamiento, definido en la sección 3.

La idea es dividir el arreglo a ordenar en dos mitades, ordenar cada mitad de forma recursiva (es decir, dividiendo cada una otra vez en la mitad, y así sucesivamente, hasta que queden pedazos de un elemento, que ya están ordenados) y luego combinar las soluciones.

Haciendo explícitas las etapas:

1. Dividir el arreglo en dos mitades.
2. Conquistar: Ordenar cada mitad usando merge sort (o sea, aplicando estas etapas a cada mitad)
3. Combinar las soluciones de cada mitad, que son subarreglos ordenados.

5.2.1. Dividir y conquistar el arreglo

Dividir el arreglo no tiene ciencia: se calcula la mitad con división entera y con eso se conocen los índices de cada segmento. La etapa de conquistar es simplemente hacer el llamado recursivo a cada mitad, porque en algún punto al dividir continuamente por la mitad se obtienen arreglos de longitud uno, que ya están ordenados.

Por ahora, para no pensar en cómo combinar las soluciones, encapsulamos ese paso en una función merge que se implementará después. Con eso, este es el pseudocódigo:

```

1 def merge_sort(nums: arreglo[int], i: int, j: int):
2     if i >= j:      } No se retorna nada porque el algoritmo es in-place; se usa el
3         return      } return para detener la función en el caso base de la recursión
4
5     k = (i + j) // 2
6
7     merge_sort(nums, i, k)
8     merge_sort(nums, k + 1, j)
9
10    merge(nums, i, k, j)

```

Necesitamos tener los índices como parámetros porque indican qué segmento del arreglo se está procesando. La invocación inicial, que resuelve el problema completo, usa los índices extremos del arreglo:

```
merge_sort(números, 0, números.longitud-1)
```

Nótese que el arreglo no se divide físicamente en cada llamado recursivo; únicamente se delimitan distintas regiones mediante índices. Esto es para que el algoritmo sea *in-place*, es decir, modifique directamente el arreglo original, sin crear y retornar uno nuevo. En cada llamado recursivo ocurre lo siguiente:

1. Si el segmento tiene longitud 1 (o 0), se alcanza el caso base y la función termina.
2. Si no, el segmento se divide conceptualmente en dos mitades y se realizan llamados recursivos para la mitad izquierda y luego la derecha. El llamado a la mitad derecha empieza después de que termina el llamado a la mitad izquierda.
3. Una vez ambas llamadas retornan, se ejecuta `combinar`, que mezcla ambas mitades ordenadas in-place dentro del rango delimitado por `ind_min` e `ind_max`.

Para arreglos largos, el caso base solo se alcanza después de muchos llamados recursivos, cuando el arreglo se ha dividido lo suficiente. Es decir: en las primeras invocaciones, siempre ocurre que el segmento se divide conceptualmente. Se acumulan llamados a `merge_sort` uno encima de otro, sin retornar ni combinar. Ese es el *descenso* de la recursión.

Cuando algún llamado alcanza el caso base, ocurre el primer retorno. La ejecución continúa justo después de ese llamado recursivo (o sea, al final de la línea 5 o de la línea 6) y entonces pasa una de dos cosas: o se realiza otro llamado recursivo (que pronto alcanzará el caso base, porque el segmento ya es muy pequeño), o se ejecuta por primera vez el paso de combinar, mezclando esos dos segmentos (que son pequeños, porque hace poco se llegó al caso base). Después de combinar, la función termina y retorna a quien la había invocado. Así, los llamados a `merge_sort` que se habían acumulado empiezan a retornar uno tras otro y la recursión comienza a *desenrollarse*. Ese es el *ascenso* de la recursión.

5.2.2. Combinar las mitades

Ahora, solo falta pensar en cómo combinar las mitades. Sorprendentemente, la mejor forma de combinarlas es la forma más intuitiva. Si cada arreglo fuera una pila de cartas, el método natural sería comparar la primera de cada pila y tomar la menor; luego comparar la siguiente de esa pila con la primera de la otra y tomar la menor (que en la combinación es la que le sigue a la primera que se había tomado), y así sucesivamente. El pseudocódigo es el siguiente:

```

1 def merge(nums: arreglo[int], i: int, k: int, j: int):
2     left = copia de nums desde i hasta k
3     right = copia de nums desde k+1 hasta j
4
5     l = 0
6     r = 0
7
8     while l < len(left) and r < len(right):
9         if left[l] <= right[r]:
10             nums[i + l + r] = left[l]
11             l += 1
12         else:
13             nums[i + l + r] = right[r]
14             r += 1
15
16     while l < len(left):
17         nums[i + l + r] = left[l]
18         l += 1
19
20     while r < len(right):
21         nums[i + l + r] = right[r]
22         r += 1

```

Se crean arreglos nuevos con cada mitad. La complejidad espacial es entonces $\Theta(n)$.

Índice en right

<= en lugar de < para estabilidad.

Lógica principal, cuando ambos subarreglos tienen elementos: se toma el menor y se añade a `nums`.

Si quedan elementos en el arreglo de la izquierda, ya no quedan en el de la derecha. Se añade el excedente de la izquierda a `nums`.

Se añade el excedente de la derecha a `nums`. Si este `while` corre, es porque el anterior no corrió y viceversa.

Un algoritmo es *estable* si mantiene el orden relativo de los elementos iguales. O sea, en caso de empate, deja primero al que estaba primero en el arreglo original. En la línea del comentario sobre estabilidad, se

elige el elemento de la izquierda en caso de empate para que continúe ubicado antes que el de la derecha en la solución final.

Práctica: 🐛 Sort an Array

5.3. Subarreglo máximo

Dado un arreglo de números, a un subarreglo contiguo no vacío cuya suma (de sus elementos) es mayor que la de cualquier otro subarreglo contiguo se le denomina un **subarreglo máximo**. El problema de encontrar un subarreglo máximo se define como:

- **Entrada:** arreglo con n números: $[a_1, a_2, \dots, a_n]$.
- **Salida:** subarreglo contiguo $[a_i, a_{i+1}, \dots, a_j]$ no vacío tal que la suma $\sum_{k=i}^j a_k$ es máxima.

“es máxima” se puede formalizar en una expresión matemática que básicamente dice que esa sumatoria es mayor que cualquier otra sumatoria de elementos contiguos que se pueda formular. Nótese que podría haber más de un subarreglo máximo: podrían haber varios cuya suma sea igual. Basta con hallar cualquiera que sea máximo.

Hay dos tipos de instancias de este problema que son muy fáciles:

- Cuando todos los valores del arreglo son positivos, la respuesta es todo el arreglo, porque todos los valores aportan a una mayor suma.
- Cuando todos los valores son negativos, se quiere que la suma sea lo menos negativa, por lo que se forma el subarreglo con un solo elemento, el menos negativo.

Para facilitar la implementación, se transforman levemente las especificaciones del problema por unas equivalentes:

- **Entrada:** arreglo con n números: $[a_0, a_1, \dots, a_{n-1}]$.
- **Salida:** índices i, j del arreglo, donde $0 \leq i \leq j < n$ tales que la suma $\sum_{k=i}^j a_k$ es máxima.

Los cambios son dos: primero, se utilizan índices basados en 0 para el arreglo (menos usual en notación matemática, pero más compatible con lenguajes de programación) y segundo, ahora se buscan los índices que delimitan el máximo subarreglo, en lugar del subarreglo como tal.

5.3.1. Solución por fuerza bruta

Una primera solución a este problema se consigue simplemente comparando todas las posibilidades de subarreglos y eligiendo el que tiene la mayor suma:

```

1 def max_subarray_brute_force(nums: arreglo[int]) -> tuple[int, int, int]:
2     largest = -∞
3
4     best_i = 0
5     best_j = 0
6
7     for i=0 to len(nums)-1:
8         total = 0
9         for j=i to len(nums)-1:
10             total += nums[j]
11             if total > largest:
12                 largest = total
13                 best_i = i
14                 best_j = j
15     return best_i, best_j, largest

```

Ese algoritmo cuenta con dos ciclos anidados que recorren el arreglo, por lo que intuitivamente su complejidad temporal es $O(n^2)$.

5.3.2. Solución por dividir y conquistar

Para diseñar un algoritmo usando dividir y conquistar, el primer paso convencional es partir el arreglo. Con eso, cada subproblema es hallar un subarreglo máximo en el respectivo pedazo. El cuidado que hay que tener con esa solución es que, al partir el arreglo, podríamos tener el infortunio de partirlo justo donde estaba el subarreglo máximo, entonces en el paso de combinar las soluciones, tenemos que asegurarnos de contemplar esa posibilidad. Con eso, las etapas son:

1. Dividir el arreglo en dos mitades.
2. Conquistar: Para cada mitad, hallar un subarreglo máximo recursivamente.
3. Combinar: Comparar los subarreglos máximos de cada mitad para elegir el mayor y también compararlos con los subarreglos que pasan por donde se dividió el arreglo.

Si encapsulamos la última parte del paso de combinar en `subarr_max_transversal`, de forma que aplazamos un poco ese problema, podemos escribir el pseudocódigo para las primeras dos etapas:

```

1 def max_subarray_divide_conquer(nums: arreglo[int], i: int, j: int) -> tuple[int, int, int]:
2     if i == j: return i, j, nums[i]
3
4     k = i + (j - i) // 2
5
6     left_i, left_j, left_sum = max_subarray_divide_conquer(nums, i, k)
7     right_i, right_j, right_sum = max_subarray_divide_conquer(nums, k+1, j)
8     cross_i, cross_j, cross_sum = max_crossing_subarray(nums, i, k, j)
9
10    if left_sum >= right_sum and left_sum >= cross_sum:
11        return left_i, left_j, left_sum
12
13    if right_sum > cross_sum:
14        return right_i, right_j, right_sum
15
16    return cross_i, cross_j, cross_sum

```

Ahora sí, pensemos los subarreglos que pasan por donde se dividió el arreglo. Todos esos subarreglos tienen un pedazo a la izquierda de esa mitad y un pedazo a la derecha. Para maximizar la suma, queremos maximizar la de la derecha y maximizar la de la izquierda. En este caso, usar un algoritmo exhaustivo no es ingenuo:

```

1 def max_crossing_subarray(nums: arreglo[int], i: int, k: int, j: int) -> tuple[int, int, int]:
2     right_sum = 0
3     left_sum = 0
4
5     largest_right = -∞
6     largest_left = -∞
7
8     for r=k to j:
9         right_sum += nums[r]
10        if right_sum > largest_right:
11            largest_right = right_sum
12            best_r = r
13
14    for l=k-1 downto i:
15        left_sum += nums[l]
16        if left_sum > largest_left:
17            largest_left = left_sum
18            best_l = l
19
20    return best_l, best_r, largest_left + largest_right

```

Ejercicios**Calentamiento**

- Longest Common Prefix

Estándar

- Kth Largest Element in an Array
- Search a 2D Matrix II

Duros

- Median of Two Sorted Arrays
- The Skyline Problem
- Reverse Pairs

6. Programación dinámica

La **programación dinámica**² es una técnica de diseño de algoritmos usualmente aplicable a problemas de optimización o de conteo.

Se puede ver como una mejora de dividir y conquistar para ciertos problemas. En ocasiones, al usar dividir y conquistar, los subproblemas en los que se dividió el problema original comparten subsubproblemas idénticos. Al resolver cada subproblema por aparte, los subsubproblemas iguales se resuelven varias veces. La idea fundamental de la programación dinámica es resolver cada subsubproblema solo una vez y almacenar su resultado en una estructura de datos para poder reutilizarlo.

En ese sentido, se realiza un *trade-off* de complejidad espacial por complejidad temporal: se gasta más espacio para un algoritmo más veloz.

6.0.1. Cuándo usar programación dinámica

No tiene sentido intentar diseñar algoritmos de programación dinámica para todos los problemas. Los problemas que se resuelven eficientemente con programación dinámica *siempre* presentan dos propiedades:

1. **Subestructura óptima:** la solución óptima del problema original se puede construir a partir de soluciones óptimas a sus subproblemas. Esto es un prerequisite para que exista un algoritmo de programación dinámica.
2. **Subproblemas superpuestos:** varios subproblemas tienen subsubproblemas repetidos, de forma que un algoritmo recursivo acabaría resolviendo los mismos subsubproblemas varias veces.
 - Para que eso ocurra, el espacio de posibles subproblemas debe ser pequeño, usualmente polinomial en el tamaño de la entrada. Si el espacio es muy grande, es probable que pocos o ningún subsubproblema se repita.
 - Puede que exista un algoritmo de programación dinámica para el problema incluso si no presenta subproblemas superpuestos, pero entonces usar programación dinámica no es más eficiente que usar dividir y conquistar o alguna solución recursiva.

6.0.2. Tipos de programación dinámica

Hay en esencia dos formas de diseñar algoritmos de programación dinámica:

- **Top-down** (del problema más general a los más pequeños): Se resuelve el problema original de forma recursiva, pero cada vez que se resuelve un subproblema, se almacena su resultado para evitar tener que recomputarlo. A eso se le llama **memoización**³.
 - En la implementación, siempre se crea una estructura de datos para almacenar los resultados de cada llamado recursivo, que inicialmente se llena con valores que indiquen que el resultado en esa

²El nombre no tiene relación con cómo funciona la técnica; fue un término acuñado por Richard Bellman en la década de 1950 para describir su investigación, deliberadamente ambiguo para evitar la aversión de la Fuerza Aérea estadounidense hacia la investigación matemática en aquel entonces.

³El término "memoización" proviene del inglés *memoization*, que significa convertir los resultados en un memo, abreviación del latín *memorandum*: algo para recordar.

posición no se ha computado aún (e.g., null, ∞ , etc.). Para que la información persista entre llamados recursivos, esa estructura tiene que estar definida afuera de la función recursiva, en otra función que invoca la recursiva; si no, cada llamado recursivo crearía su propia estructura, sin los resultados previos.

- **Bottom-up** (de los problemas pequeños al general): Se resuelven primero los subproblemas más pequeños y se almacenan sus resultados en una tabla. A eso se le denomina **tabulación**. Se usan esos resultados para resolver subproblemas más grandes, hasta llegar al problema original.
 - Como los subproblemas se resuelven en orden de tamaño, se reemplaza el uso de recursión por un ciclo.

Como ambos métodos aprovechan la propiedad de subproblemas superpuestos, sus complejidades asintóticas suelen ser iguales. Sin embargo, cuando *todos* los subproblemas posibles se tienen que resolver al menos una vez, el algoritmo bottom-up suele ser más eficiente que el top-down por un factor constante, porque el algoritmo top-down tiene el sobrecosto de la recursión (véase el apéndice ??).

Ocasionalmente, no es necesario resolver todos los subproblemas del espacio, solo algunos; en ese caso, la solución bottom-up puede ser peor porque requiere resolver los subproblemas pequeños primero para completar la tabulación, mientras que la solución top-down solo memoiza los subproblemas que sí se tienen que resolver.

6.0.3. Recomendaciones

La programación dinámica es ampliamente considerada una de las técnicas de diseño de algoritmos más difíciles. En esta sección se propone el siguiente método para afrontarla:

1. Entender el problema, que es *sine qua non* para solucionar cualquier cosa.
2. Identificar una forma de subestructura óptima para el problema.
 - Como se mencionó arriba: si no existe una subestructura óptima para el problema, no se puede resolver con un algoritmo de programación dinámica.
 - Un problema puede tener varias formas de plantear una subestructura óptima, que llevan a algoritmos de programación dinámica distintos.
 - Se puede escribir la subestructura óptima como una **relación de recurrencia**: una regla que expresa un término de una secuencia en función de términos anteriores.
 - Si se encuentra una relación de recurrencia correcta para el problema, es garantía de subestructura óptima.
 - Toda relación de recurrencia se puede escribir como una ecuación recursiva.
3. Determinar si presenta subproblemas superpuestos
 - Si, en el paso anterior, se formuló la relación de recurrencia, se puede escribir como ecuación recursiva y resolverse
4. Cuántas variables
5. Grafo de necesidades
6. DP
7. Backtracking

A continuación se presentan múltiples ejemplos de problemas que se resuelven eficientemente usando programación dinámica, ordenados en dificultad ascendente.

6.1. Sucesión de Fibonacci

Recuérdese la definición de la sucesión de Fibonacci, como

$$\begin{aligned} F(0) &= 0 \\ F(1) &= 1 \\ F(n) &= F(n-1) + F(n-2) \quad \text{para } n > 1 \end{aligned}$$

De entrada se puede ver la subestructura óptima en la fórmula: la solución óptima, $F(n)$, se construye a partir de las soluciones óptimas para dos subproblemas, $F(n-1)$ y $F(n-2)$. Con eso en mente, será viable utilizar programación dinámica.

6.1.1. Subproblemas superpuestos

No es difícil ver que el algoritmo anterior presenta subproblemas superpuestos: para calcular $F(n)$ se necesitan calcular $F(n-1)$ y $F(n-2)$; para calcular $F(n-1)$ se necesitan calcular $F(n-2)$ y $F(n-3)$. En tan solo dos pasos, ya se repitió el cálculo de $F(n-2)$ dos veces. Eso implica que el problema se puede resolver eficientemente usando programación dinámica.

La existencia de problemas superpuestos se ve reflejada también en que el costo computacional del algoritmo es muy elevado. Dicho costo se puede formular con la siguiente función T :

$$T(n) = c + T(n-1) + T(n-2)$$

Al resolver esa ecuación recursiva (véase el Apéndice C), se obtiene que $T \in \Theta(\phi^n)$, donde $\phi \approx 1,618$ es el número áureo. Ergo, el costo computacional es exponencial.

6.1.2. Solución top-down

Una primera optimización es utilizar programación dinámica top-down: se introduce una función principal, `fibonacci_top_down`, que crea un arreglo donde se memoizarán las respuestas. Esa función principal llena el arreglo con -1 (que indica que esa solución no se ha calculado) y se lo pasa a la función auxiliar, que es la función recursiva anterior, con la diferencia de que memoiza el cálculo si aún no está en el arreglo y retorna la solución memoizada.

```

1 def fibonacci_top_down(n: int) -> int:
2     memo = arreglo de enteros de tamaño n + 1
3     memo[0] = 0      } Se añaden los casos base
4     memo[1] = 1      } directamente al arreglo
5
6     for i=2 to n:    } Se llena el resto del arreglo con -1, que indica que no se ha calculado ese término
7         memo[i] = -1 } de la secuencia. Es inambiguo porque los términos calculados serán positivos
8
9     return _fibonacci_memo(n, memo) .....> La función auxiliar recursiva recibe siem-
                                           pre la misma estructura y la va llenando
10
11 def _fibonacci_memo(n: int, memo: arreglo[int]) -> int:
12     if memo[n] < 0: .....> La estructura aún no contiene la solución para este valor, entonces, se memoiza
13         memo[n] = (
14             _fibonacci_memo(n - 1, memo) +
15             _fibonacci_memo(n - 2, memo)
16         )
17     return memo[n] .....> Se retorna siempre el valor memoizado (que puede que se
                                           haya calculado en ese llamado recursivo o en uno anterior)

```

Por definición, para calcular el n -ésimo número de Fibonacci es necesario calcular todos los anteriores, por ende, el algoritmo top-down es menos eficiente que el algoritmo bottom-up: ambos calculan todos los términos anteriores, pero el bottom-up evita el uso de recursión.

6.1.3. Solución bottom-up

El método con programación dinámica bottom-up resuelve primero los problemas más pequeños. En este caso, eso traduce naturalmente a calcular primero los menores términos de la sucesión, o lo que es lo mismo, a calcularlos en orden:

```

1 def fibonacci(n: int) -> int:
2     tab = arreglo de int de tamaño n + 1
3     tab[0] = 0
4     tab[1] = 1
5
6     for i=2 to n:
7         tab[i] = tab[i-1] + tab[i-2]
8
9     return tab[n]
```

Se obtiene un algoritmo cuya complejidad temporal es $\Theta(n)$.

Este problema tiene el beneficio de que solo se necesitan las últimas dos soluciones para computar la próxima. Por ende, la tabulación se puede realizar empleando solo dos variables en lugar de un arreglo. Eso hace que la complejidad espacial no dependa del tamaño de la entrada, reduciéndola de $\Theta(n)$ a $\Theta(1)$.

```

1 def fibonacci(n: int) -> int:
2     if n <= 1: return n
3
4     prev_prev_sol = 0
5     prev_sol = 1
6
7     for i=2 to n:
8         solution = prev_sol + prev_prev_sol
9         prev_prev_sol = prev_sol
10        prev_sol = solution
11
12    return solution
```

Práctica: 🐞 Fibonacci Number

6.2. Problema de corte de vara

El problema de corte de vara consiste en determinar cuál es la mejor forma de cortar una vara de longitud n , si cada posible longitud entera se puede vender a un precio específico. El objetivo es maximizar los ingresos.

Las entradas del problema son entonces una longitud entera n y una función p que determina el precio que tendría cada pedazo. Para facilidad computacional, usualmente el problema se establece como:

- Entradas: Un entero n que representa la longitud de la vara y un arreglo p de tamaño $n + 1$, donde $p[i]$ es el precio de una vara de longitud i .
- Salida: Los ingresos máximos que se pueden obtener al cortar la vara.

En ese modelo, $p[0] = 0$ porque longitud 0 representa la ausencia de vara (y por ende de venta).

Considérese la instancia del problema con $n = 4$ y $p = [0, 1, 5, 8, 9]$. En ese caso:

- Si no se corta la vara, se vende a $p[4]$, o sea a \$9.
- Si se corta en un pedazo de longitud 3 y otro de longitud 1, se venden a \$8 y \$1, respectivamente, para obtener ingresos de \$9.
- Si se corta en dos pedazos de longitud 2, se venden a \$10.
- Si se corta en pedazos de longitud 2, 1 y 1, se venden a \$7.

- Si se corta en cuatro pedazos de longitud 1, se venden a \$4.

Entonces, la mejor forma de cortar la vara es en dos pedazos de longitud 2, para obtener ingresos de \$10. Como n es pequeño, esta instancia se resuelve por fuerza bruta, mirando todas las formas de cortar la vara y calculando los ingresos para cada una.

Para un valor arbitrario de n , hay muchas formas de cortar la vara. En cada unidad de longitud, hay dos alternativas: se corta o no. Entonces hay 2 posibilidades para cada una de las $n - 1$ unidades de longitud ($n - 1$ porque en la longitud n ya no se puede cortar), lo que da $\underbrace{2 \cdot 2 \cdots 2}_{n-1 \text{ veces}} = 2^{n-1}$ combinaciones totales.

El conteo anterior no contempla que algunas combinaciones están repetidas, porque el orden de los pedazos no importa: es lo mismo cortar en la primera unidad de longitud y obtener un pedazo de longitud 1 y otro de longitud $n - 1$, que en la penúltima unidad de longitud, obteniendo un pedazo de longitud $n - 1$ y otro de longitud 1. Esta observación tiene dos implicaciones:

- Primero, reduce el número de combinaciones de 2^{n-1} a un valor dado por la función de partición de enteros, $e^{\pi \sqrt{2n/3}} / (4n\sqrt{3})$. Esta es una gran mejora, pero la cantidad de combinaciones sigue lejos de ser polinomial. (No se muestra la deducción de la función de partición porque es un problema de conteo mucho más difícil).
- Segundo, implica que no se debe pensar en cortar o no en cada unidad, sino que se puede pensar en cortar de forma que los pedazos queden en tamaño no decreciente. Eso igual va a contemplar todas las combinaciones: no contempla $(n - 2, 2)$, ni $(n - 2, 1, 1)$; pero sí $(2, n - 2)$, también $(1, 1, n - 2)$, etc., y como dan los mismos ingresos, con eso basta.

6.2.1. Solución recursiva

Para diseñar una solución recursiva, lo primero que hay que pensar es cómo dividir este problema en subproblemas. En este caso es relativamente fácil, porque dada una vara de longitud n , se querría evaluar:

- El primer caso de cortar en la primera unidad de longitud. Eso arroja un pedazo de longitud 1 y otro de longitud $n - 1$. Ese último pedazo de longitud $n - 1$ se puede seguir cortando (si n es grande) y entonces se tiene el mismo problema: cuál es la forma de cortarlo para maximizar los ingresos.
- El segundo caso es cortar en la segunda unidad de longitud y es análogo: el pedazo de longitud 2 se vende a $p[2]$, pero falta evaluar cómo cortar el pedazo de longitud $n - 2$ de forma óptima.

Con eso es fácil construir una implementación recursiva, `rod_cut_rec` que evalúe el primer caso como `p[1] + rod_cut_rec(n-1, p)`; el segundo, como `p[2] + rod_cut_rec(n-2, p)` y así hasta `p[n] + rod_cut_rec(0, p)`, donde ese último llamado recursivo evidentemente es un caso base.

```

1 def rod_cut_rec(n: int, prices: arreglo[int]) -> int:
2     if n == 0: return 0
3
4     optimal_rev = -∞
5     for i=1 to n:
6         revenue = prices[i] + rod_cut_rec(n-i, prices)
7         if revenue > optimal_rev:
8             optimal_rev = revenue
9
10    return optimal_rev

```

El raciocinio que usamos para esta implementación deja claro que el problema cuenta con subestructura óptima. Falta ver si también presenta subproblemas superpuestos, pues solo en ese caso un algoritmo de programación dinámica mejoraría la eficiencia.

6.2.2. Solución top-down

Para eliminar la redundancia que introducen los problemas superpuestos, se implementa un algoritmo top-down. Se añade una función principal que crea un arreglo donde la función auxiliar recursiva memoiza el

cálculo del mejor precio para cada longitud.

```

1 def rod_cut_top_down(n: int, prices: arreglo[int]) -> int:
2     optimal_revs = arreglo de int de longitud n + 1
3     optimal_revs[0] = 0
4     for i=1 to n:
5         optimal_revs[i] = -∞
6
7     return rod_cut_memo(n, prices, optimal_revs)
8
9 def rod_cut_memo(n: int, prices: arreglo[int], optimal_revs: arreglo[int]) -> int:
10    if optimal_revs[n] >= 0:
11        return optimal_revs[n]
12
13    optimal_rev = -∞
14    for i=1 to n:
15        revenue = prices[i] + rod_cut_memo(n - i, prices, optimal_revs)
16        if revenue > optimal_rev:
17            optimal_rev = revenue
18
19    optimal_revs[n] = optimal_rev
20    return optimal_rev

```

Se crea el arreglo lleno de $-\infty$ (salvo el caso base) para indicar las soluciones que aún no se han memoizado.

Si el valor está memoizado, se retorna enseguida para no recomputarlo.

Justo antes de retornar el valor, se memoiza para los llamados siguientes.

6.2.3. Solución bottom-up

Para diseñar el algoritmo bottom-up se debe reemplazar la recursión del algoritmo top-down por un ciclo.

En el algoritmo top-down, los llamados recursivos a la función auxiliar `rod_cut_memo`, con distintos valores como argumento para `n`, representan los subproblemas: cada llamado evalúa cómo cortar una posible longitud de la vara (dada por un valor distinto de `n`).

Como en la solución bottom-up no se invocan funciones auxiliares, el valor de `n` que entra como parámetro no cambia. Por tanto, el ciclo que hace falta introducir, que reemplaza la recursión, es uno que revisa todos los posibles valores de `n`, es decir todos los subproblemas, y para cada uno de ellos sigue la misma lógica que la función auxiliar de la versión top-down. Con eso, la solución bottom-up acaba con dos ciclos: uno externo, que toma el lugar de la recursión, y uno interno, idéntico al ciclo único de los algoritmos previos.

```

1 def rod_cut_bottom_up(n: int, prices: arreglo[int]) -> int:
2     optimal_revs = arreglo de int de longitud n + 1
3     optimal_revs[0] = 0
4
5     for subprob_len=1 to n:
6         optimal_rev = -∞
7         for i=1 to subprob_len:
8             revenue = prices[i] + optimal_revs[subprob_len-i]
9             if revenue > optimal_rev:
10                 optimal_rev = revenue
11
12         optimal_revs[subprob_len] = optimal_rev
13
14    return optimal_revs[n]

```

Como el arreglo se llenará en orden, no hay que llenarlo de antemano como en top-down, con el primer elemento basta.

En top-down, la longitud `n` cambia en cada llamado recursivo para cada subproblema; acá, `n` nunca cambia, entonces `subprob_len` toma su rol como la longitud de cada subproblema.

El ciclo interno es calcado de las versiones anteriores, pero la tabulación reemplaza la recursión.

Esa solución cuenta con complejidad temporal de $\Theta(n^2)$, por los dos ciclos anidados, y con complejidad espacial de $\Theta(n)$, por el arreglo que se utiliza para la tabulación.

6.2.4. Obtener los tamaños de los cortes

Inicialmente, el problema de corte de vara se planteó como un problema de optimización, donde la respuesta esperada era el máximo ingreso posible. Sin embargo, el problema se podría plantear como un problema de

búsqueda en el que se busca alguna configuración de cortes que maximiza el ingreso, en lugar del ingreso en sí.

- Entradas: Un entero n que representa la longitud de la vara y un arreglo p de tamaño $n + 1$, donde $p[i]$ es el precio de una vara de longitud i .
- Salida: Una pila con las longitudes de las piezas que maximizan el precio de venta.

Para esto, al implementar el algoritmo de programación dinámica se debe no solo computar el valor óptimo sino también almacenar información para que, una vez se obtenga el valor óptimo, se pueda recuperar los pasos que se siguieron para obtenerlo.

En este caso, además de almacenar el precio óptimo para una longitud ℓ , se almacena el primer corte que corresponde a ese precio óptimo. Solo se almacena el primer corte c para cada longitud, porque para determinar posteriormente el resto de cortes (si se requieren más de un corte), basta con examinar el subproblema de la longitud restante, o sea, el subproblema de la vara de longitud $\ell - c$ (para el cual también se guardó el primer corte).

```

1 def rod_cut(n: int, prices: arreglo[int]) -> pila[int]:
2     optimal_revs = arreglo de int de longitud n + 1
3     optimal_revs[0] = 0
4     optimal_cuts = arreglo de int de longitud n + 1
5
6     for subprob_len=1 to n:
7         optimal_rev = -∞
8         for i=1 to subprob_len:
9             revenue = prices[i] + optimal_revs[subprob_len-i]
10            if revenue > optimal_rev:
11                optimal_rev = revenue
12                optimal_cut = i
13            optimal_revs[subprob_len] = optimal_rev
14            optimal_cuts[subprob_len] = optimal_cut
15
16     segments = pila de int de longitud máxima n
17     i = n
18     while i > 0:
19         cut = optimal_cuts[i]
20         segments.put(cut)
21         i -= cut
22
23     return segments

```

Se almacena el primer corte que se hizo en la solución óptima de ese subproblema.

Para una longitud ℓ , el primer corte es $\text{cortes_optimos}[\ell]$; los cortes restantes (si hay) se deben averiguar mirando el subproblema de la longitud restante, que es $\ell - \text{cortes_optimos}[\ell]$

Se toma el primer corte óptimo para n ; se resta de n para obtener una nueva longitud, luego se mira el primer corte para ella; y así hasta acabar la vara

Práctica: ☞ Rod Cutting

6.3. Multiplicación de matrices en cadena

La multiplicación de matrices en cadena es un ejemplo clásico de optimización de **planes de ejecución**, en donde el orden en el que se ejecutan los pasos para completar una tarea influye drásticamente en su costo computacional. En este caso, el orden en el que se multiplica cada par de matrices hace que se requieran muchas más o muchas menos multiplicaciones entre escalares.

6.3.1. Costo computacional



Tip. Si no recuerda las peculiaridades de la multiplicación de matrices, lea antes el Apéndice D.

Primero, miremos cuántas operaciones se necesitan para multiplicar dos matrices. Para $A_{p \times q} B_{q \times r}$, el resultado será de tamaño $p \times r$, donde cada uno de los pr elementos se computa mediante un producto punto

entre vectores de tamaño q . En cada producto punto se realizan entonces q multiplicaciones entre escalares, para un total de pqr productos entre escalares.

Para una multiplicación en cadena de tres matrices:

- Si se multiplica agrupando las matrices como $(\mathbf{A}_{p \times q} \mathbf{B}_{q \times r}) \mathbf{C}_{r \times s}$ (que es equivalente a no agrupar, se multiplica de izquierda a derecha), el costo computacional está dado por pqr del producto $(\mathbf{AB})_{p \times r}$, al que se le suma prs de la multiplicación entre ese producto y $\mathbf{C}_{r \times s}$, acumulando

$$pqr + prs = pr(q + s).$$

Esa expresión es pequeña cuando p y r son pequeños comparados con q y s .

- Si realiza el producto con la otra agrupación posible, $\mathbf{A}_{p \times q}(\mathbf{B}_{q \times r} \mathbf{C}_{r \times s})$, se cuentan qrs operaciones dentro del paréntesis, sumadas a pqs cuando se multiplica $\mathbf{A}$ por eso, para un total de

$$pqs + qrs = qs(p + r)$$

Esta agrupación es preferible cuando q y s son pequeños relativos a p y r .

A partir de ese análisis es fácil fabricar ejemplos que ilustran cuán distintos son los costos computacionales con tres matrices de tamaños concretos: para el producto $\mathbf{A}_{100 \times 3} \mathbf{B}_{3 \times 150} \mathbf{C}_{150 \times 2}$, computar $\mathbf{ABC}$ requiere de $100 \cdot 150(3 + 2) = 75000$ multiplicaciones escalares, mientras que computar $\mathbf{A(BC)}$ requiere de tan solo $3 \cdot 2(100 + 150) = 1500$ multiplicaciones, ¡50 veces menos!

A pesar de que el análisis anterior es útil para dimensionar por qué es importante agrupar las matrices inteligentemente, es un análisis que no es escalable: a medida que el número de matrices a multiplicar incrementa, la cantidad de agrupaciones completas crece de forma exponencial en función del número de matrices y decidir con base en fórmulas se torna inviable. Para 3 matrices solo hay 2 formas de agruparlas completamente; para 4 matrices, ya hay 5 formas:

1. $((\mathbf{AB})\mathbf{C})\mathbf{D}$, equivalente a $\mathbf{ABCD}$
2. $(\mathbf{A(BC)})\mathbf{D}$, equivalente a $\mathbf{A(BC)D}$
3. $\mathbf{A((BC)D)}$, equivalente a $\mathbf{A(BCD)}$
4. $(\mathbf{AB})(\mathbf{CD})$, equivalente a $\mathbf{AB(CD)}$
5. $\mathbf{A(B(CD))}$

En general, para n matrices hay $\frac{2n!}{(n+1)!n!}$ posibles agrupaciones completas. Ese cálculo corresponde a la sucesión de números de Catalán, cuya deducción es poco trivial, por lo que se desglosa en el Apéndice E. El caso es que, enfrentados a la imposibilidad de elegir la mejor agrupación empleando solo álgebra, es hora de diseñar algoritmos.

6.3.2. Subestructura óptima

Una forma de ver la subestructura óptima en este problema es pensarlo desde el final hacia el principio. Independientemente de cómo se hayan agrupado las matrices para multiplicarlas, la última multiplicación tendrá que ser entre dos matrices. Ambas matrices vienen de multiplicar varias otras; si la primera es el producto de k matrices, la segunda es el producto de $n - k$ matrices: $(\mathbf{A}_1 \cdots \mathbf{A}_k)(\mathbf{A}_{k+1} \cdots \mathbf{A}_n)$. Esa última expresión generaliza cualquier agrupación si $1 \leq k < n$ y la abreviamos $\mathbf{A}_{1..k} \mathbf{A}_{k+1..n}$.

A partir de esa idea, la subestructura óptima radica en que, para que ese producto $\mathbf{A}_{1..k} \mathbf{A}_{k+1..n}$ sea la agrupación óptima, entonces ambos factores $\mathbf{A}_{1..k}$ y $\mathbf{A}_{k+1..n}$ deben estar, a su vez, agrupados óptimamente. Eso se debe a que si alguno de los factores se pudiera agrupar mejor, entonces bastaría con reemplazarlo por esa mejor agrupación para obtener una agrupación final mejor que la óptima, lo cual es una contradicción. Así pues, todo subproblema tiene la forma $\mathbf{A}_{i..k} \mathbf{A}_{k+1..j}$ para algunos valores $1 \leq i \leq k \leq j < n$.

6.3.3. Formulación recursiva

Con base en lo anterior, se puede formular una solución recursiva. Cada subproblema $\mathbf{A}_{i..k}\mathbf{A}_{k+1..j}$ solo es interesante si $i < j$; cuando $i \geq j$ solo hay una (o ninguna) matriz. Para evaluar cuál agrupación $\mathbf{A}_{1..k}\mathbf{A}_{k+1..n}$ tiene menor costo, se debe buscar la de menor costo considerando cada posible k entre i y j .

Recuérdese que el costo está dado por: el costo de las multiplicaciones $\mathbf{A}_{1..k}$, más el de las multiplicaciones $\mathbf{A}_{k+1..n}$, sumados al costo de multiplicar $\mathbf{A}_{1..k}\mathbf{A}_{k+1..n}$. Para formular ese último costo, se necesita las dimensiones de las matrices, entonces se va a denotar el tamaño de la k -ésima matriz como $p_k \times p_{k+1}$, de forma que el producto queda como: $(\mathbf{A}_{1 \times p_2} \cdots \mathbf{A}_{p_k \times p_{k+1}})(\mathbf{A}_{k+1 \times p_{k+2}} \cdots \mathbf{A}_{p_n \times p_{n+1}})$

Con todo eso en mente, sea $f(i, j)$ el costo óptimo de multiplicar las matrices entre la i y la j :

$$f(i, j) = \begin{cases} 0 & \text{si } i \geq j \\ \min_{i \leq k < j} f(i, k) + f(k+1, j) + p_i p_{k+1} p_{j+1} & \text{si } i < j \end{cases}$$

7. Algoritmos aproximados

Un **algoritmo aproximado** arroja soluciones cercanas a la óptima. A causa de eso, los algoritmos aproximados son incorrectos, pero resultan útiles para problemas en los que no se conocen algoritmos correctos eficientes. En la práctica, cuasi optimalidad suele bastar para muchas aplicaciones, sobretodo si el algoritmo ejecuta en tiempo polinomial para el caso de uso esperado; en ese caso, suele preferirse a un algoritmo correcto pero no polinomial.

7.1. Razón de aproximación

Para determinar la calidad de un algoritmo aproximado se busca determinar qué tan cercana es la solución que produce a la solución óptima.

Para un problema de maximización cuya solución óptima tiene un valor $C^* > 0$, si un algoritmo aproximado produce una solución con valor $C > 0$, el factor por el cual la óptima es mejor que la aproximada es la razón $\frac{C^*}{C}$. Entre mayor sea el factor, peor es la aproximación: si el factor es 1, las soluciones son igual de buenas; si es 1.5, la solución óptima es 50 % mejor que la aproximada. La razón no puede ser menor que 1 porque no hay soluciones mejores que la óptima, $0 < C \leq C^*$.

Similarmente, si el problema es de minimización, $C \geq C^* > 0$ y la razón $\frac{C}{C^*} \geq 1$ indica cuánto mejor es la óptima de la aproximada. En general, si no se sabe si el problema de optimización busca el máximo o el mínimo, tomar $\max\{\frac{C^*}{C}, \frac{C}{C^*}\}$ elige automáticamente la proporción mayor que 1, que es la que tiene sentido. Con base en esa idea, se define la razón de aproximación.

Definición 7.1. Razón de aproximación

Un algoritmo tiene una **razón de aproximación** de $\rho(n)$ si para cualquier tamaño de entrada n se cumple que

$$\max\left\{\frac{C}{C^*}, \frac{C^*}{C}\right\} \leq \rho(n).$$

La razón de aproximación es entonces una cota superior de la calidad de la aproximación: en el peor caso, la solución del algoritmo está dentro de un factor $\rho(n)$ de una solución óptima. Puede pasar que la aproximación empeora conforme el tamaño de la entrada aumenta; si, por el contrario, $\rho(n)$ es constante, la calidad de la aproximación no depende de la entrada.

Los algoritmos aproximados se categorizan con base en su razón de aproximación; se dice que el algoritmo es de aproximación $\rho(n)$.

7.2. Problema de vertex-cover

Un cubrimiento por vértices (*vertex cover*) de un grafo (V, E) es un subconjunto de vértices $S \subseteq V$ que cubre todas las aristas, es decir, que toda arista $(u, v) \in E$ tiene al menos un extremo en S : $u \in S$ o $v \in S$.

El **problema de vertex-cover** consiste en hallar un cubrimiento por vértices de tamaño mínimo.

7.2.1. Algoritmo aproximado para vertex-cover

A pesar de que nadie ha hallado un algoritmo polinomial correcto, una idea bastante intuitiva resulta en un algoritmo de aproximación 2.

La idea es: mientras queden aristas sin cubrir, se elige arbitrariamente una de ellas, se agregan ambos extremos a la solución y se descartan todas las demás aristas incidentes a esos extremos.

```

1 def aprox_vertex_cover(G: grafo) -> set[vértices]:
2     S = ∅
3     E = copia de las aristas de G
4     while E ≠ ∅:
5         e = eje arbitrario (u, v) ∈ E
6         S = S ∪ {u, v}
7         eliminar cada arista incidente a u o v de E
8     return S

```

7.2.2. Complejidad temporal

7.2.3. Razón de aproximación

Parte III

Análisis de algoritmos

Analizar un algoritmo consiste en determinar si es correcto y predecir qué recursos consumiría.

El análisis es importante porque uno quiere que su algoritmo sea correcto y rápido. Eso suele ser difícil; a menudo hay algoritmos correctos pero lentos o algoritmos rápidos pero probabilísticos.

La sección 8 se ocupa de la correctitud, mientras que la sección 9 se ocupa del tiempo de ejecución (y expone conceptos que se pueden usar para analizar otros recursos, de ser necesario).

8. Correctitud

Definición 8.1. Correctitud

Un algoritmo es **correcto** si para toda instancia de entrada arroja la salida correcta, es decir, la salida que satisface la relación con la entrada definida por el problema.

Como la correctitud se debe verificar para todas las posibles instancias, usualmente exige una demostración matemática.

9. Eficiencia

Entre dos algoritmos correctos que resuelven un mismo problema, naturalmente se prefiere el que menos recursos usa: el más **eficiente**.

Usualmente, el recurso máspreciado es el tiempo, por lo cual el análisis de eficiencia suele estar enfocado en el tiempo de ejecución; sin embargo, se pueden seguir principios similares para analizar otros recursos, como el uso de memoria.

9.1. Complejidad temporal

Dado un algoritmo, (usualmente) es posible realizar una **implementación**: escribir un programa que ejecuta el algoritmo. El tiempo de ejecución de una implementación de un algoritmo depende de:

1. La máquina en la que se ejecuta.
2. El lenguaje de programación en el que se implementa.
3. El número de operaciones que realiza el algoritmo.
4. Los datos que recibe el programa como entrada.

Al analizar un algoritmo, se pretende sacar conclusiones generales; por ende, no basta con implementar el algoritmo en una máquina y lenguaje específicos para medir su tiempo de ejecución. Como solución, se intenta contar el número de operaciones elementales que realiza el algoritmo. Para eso, en lugar de ejecutar el algoritmo en una máquina real, se analiza el algoritmo en el marco de un **modelo de computación** que abstrae la máquina física y define cuáles operaciones son elementales.

Usualmente, la cantidad de operaciones depende de qué tantos datos el algoritmo recibe como entrada. Por ello, la cantidad se expresa como una función matemática del tamaño de la entrada. Esa función se denomina **costo computacional temporal**.

Se puede analizar el crecimiento asintótico de esa función para saber cómo escala el costo del algoritmo a medida que aumenta el tamaño de la entrada. El resultado de ese análisis es la **complejidad temporal** del algoritmo, y suele ser el objetivo último del análisis de eficiencia.

9.1.1. Modelo de computación RAM

Un modelo de computación es una abstracción de la máquina en la que se ejecuta un algoritmo, que especifica cuáles son las operaciones primitivas disponibles y cuál es el costo asociado a cada una.

Para analizar algoritmos, tiene sentido utilizar un modelo de computación que represente razonablemente bien a los computadores convencionales modernos. Un modelo de computación apropiado es el **modelo RAM de costo uniforme**, (*Random Access Machine*, máquina de acceso aleatorio⁴) que establece las siguientes propiedades:

- El modelo describe un único procesador secuencial.
 - Las instrucciones se ejecutan una tras otra, sin concurrencia ni paralelismo.
- La memoria interna es direccionable aleatoriamente y es infinita.
 - Se puede acceder a cualquier posición de memoria directamente si se conoce su dirección, sin tener que recorrer todas las posiciones anteriores. Ergo, cada acceso a memoria se realiza en un tiempo constante.
 - No hay de restricciones físicas de almacenamiento, no preocupa agotar la memoria.
- El tamaño de palabra es finito y suficiente para almacenar los datos manipulados por operaciones básicas.
- Toda instrucción elemental (aritmética básica, comparación, asignación, salto y acceso a memoria) se realiza en tiempo constante.

Para ser formales, hace falta listar explícitamente qué se considera una instrucción elemental. Sin embargo esa labor es engorrosa, requiere conocimientos en arquitectura de computadores y aporta poco. Por lo pronto, basta con tener presente que la idea del modelo RAM es abstraer un computador real, por lo que las instrucciones simples en el modelo RAM son aquellas que son simples en un computador normal. Los bucles y las subrutinas, por ejemplo, no son instrucciones primitivas en un computador común, por lo que tampoco lo son en el modelo RAM.

Existen otros muchos modelos de computación, que pretenden representar otras cosas: máquinas de Turing; manejo de memoria moderno con memoria externa, memoria virtual y caché; procesadores paralelos; números de gran tamaño; entre otros. Incluso se puede usar un modelo RAM de costo variable, que contempla que las instrucciones elementales toman un tiempo proporcional al tamaño de los operandos. No obstante, en el análisis clásico de algoritmos ninguno de esos factores suele considerarse y por tanto se da por sentado el modelo RAM de costo uniforme.

9.2. Análisis exacto

El análisis exacto de un algoritmo consiste en:

1. Asignar un tiempo constante a cada instrucción elemental del algoritmo
2. Contar cuántas veces ocurre cada instrucción
3. Sumar esos tiempos

A eso se le denomina **análisis exacto** de la eficiencia.

⁴No confundir este uso de la abreviación RAM como el modelo de computación de máquina de acceso aleatorio con el uso que se le da en otros contextos de informática como la memoria de acceso aleatorio.

10. Intratabilidad

Los problemas que se pueden resolver con un computador se clasifican de acuerdo a la complejidad temporal del mejor algoritmo conocido para resolverlos. Eso da una noción de su dificultad: si alguien ha encontrado un algoritmo polinomial para el problema, se dice que es **tratable**⁵; si nadie lo ha hecho, de forma que solo se conocen algoritmos de tiempo súperpolinomial, se dice que es **intratable**.

Otra forma de comparar problemas es a través de reducciones. Una **reducción** consiste en transformar un problema en otro, tal que resolver ese otro te permita resolver el original.

Definición 10.1. Reducción

Un problema A se puede **reducir** a otro problema B si existe un algoritmo que, para cualquier instancia de A , construya una instancia de B tal que la solución de esa instancia de B permita construir la solución de la instancia original de A .

Si existe una reducción de A a B y esa reducción es polinomial, el algoritmo que resuelve B ahora se puede utilizar para resolver A y entonces se dice que B es igual o más difícil que A . Usualmente, cuando se habla de una reducción, se presume que es polinomial, porque de lo contrario no es útil: si la reducción es más compleja que resolver el problema, no sirve de nada transformarlo.

Con eso, se describe la clasificación formal de problemas, aunque solo más adelante se dan definiciones genuinamente rigurosas para cada clase de problemas. La clasificación es la siguiente:

- P (**polinomial**): problemas para los que se conoce algún algoritmo polinomial correcto que los resuelva.
- NP (**no determinista polinomial**): problemas para los que se conoce algún algoritmo polinomial que, dado un candidato de solución, verifique si es correcto.
 - Como todos los problemas que están en P se resuelven polinomialmente, también están en NP, porque también se pueden verificar polinomialmente. Es decir, $P \subseteq NP$.
 - Puede ser que todos los problemas que se pueden verificar en tiempo polinomial también se puedan resolver en tiempo polinomial, es decir, que $P = NP$, pero no se ha demostrado ni refutado esa posibilidad. A esta interrogante se le llama **P vs NP**⁶.
- NP-Hard (**no determinista polinomial difícil**): problemas tal que cualquier problema en NP se puede reducir en tiempo polinomial a ellos.
 - Como cualquier problema en NP se puede reducir polinomialmente a ellos, se dice que son igual o más difíciles que cualquier problema NP.
 - No necesariamente los problemas NP-Hard son NP; muchos no se pueden verificar en tiempo polinomial.
 - Si mañana se encuentra un algoritmo polinomial para resolver algún problema NP-Hard, sería revolucionario: cada problema NP se puede reducir polinomialmente a ese problema NP-Hard, que ahora podría solucionarse polinomialmente con ese nuevo algoritmo. Eso implicaría que todo problema NP sería polinomial, o sea que $P = NP$.
- NP-Completo (**no determinista polinomial completo**): problemas NP-Hard que son NP.
 - Como son NP-Hard, para ninguno se conoce un algoritmo polinomial, pero como son NP, se pueden verificar en tiempo polinomial. En ese sentido, son los problemas más difíciles de NP.
 - Son los más interesantes porque muchos problemas NP Completos son muy similares a problemas que sabemos que son P. Entonces es muy tentador intentar encontrar un algoritmo polinomial

⁵Se usa la palabra tratable como un calco de la palabra inglesa **tractable**, cuya traducción es más cercana a “manejable”

⁶El Clay Mathematics Institute ofrece un premio de un millón de dólares a quien resuelva P vs NP, como parte de sus *Millennium Prize Problems*

para alguno y concluir que $P = NP$.

Resulta evidente que las relaciones de contención entre esas clases dependen de P vs NP . La figura 1 muestra la relación entre las clases de problemas en ambos escenarios.

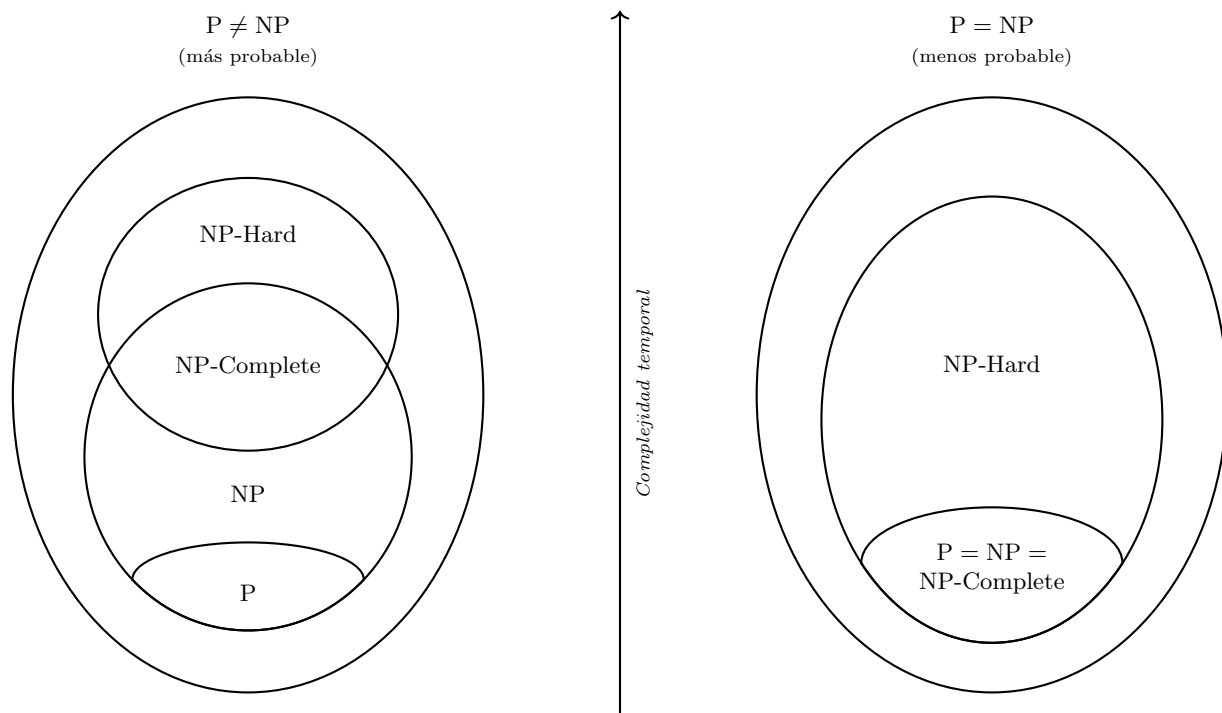


Figura 1: Clasificación de los problemas computables por clases de complejidad, para los dos escenarios del problema abierto P vs. NP .

10.1. P vs NP

Muchos de los problemas NP -Hard y NPC se han estudiado al detalle por décadas, muchas veces individualmente, y nadie ha encontrado un algoritmo polinomial para ninguno. El hecho de que encontrar un algoritmo polinomial para alguno implica que siempre han existido algoritmos polinomiales para todos, implica que sería muy sorprendente que $P = NP$. Sin embargo, hay que contemplar esa posibilidad, por remota que sea.

Para hacer más énfasis en lo anterior, imagine que muchas personas han buscado duendes en su jardín durante años, y nadie ha encontrado ninguno. Eso sugiere que no existen, pero ahora imagine un supuesto más fuerte aún: que si alguien encuentra un duende en su jardín, significa que siempre habrían habido duendes en cada jardín del mundo, pero nadie los había encontrado nunca. Eso significa no solo que quien encontró el duende tuvo gran habilidad o que su jardín era especial, sino también que todos los demás estuvieron rodeados de duendes todo el tiempo sin darse cuenta. Por eso es que se dice que es muy improbable que $P = NP$.

10.2. Problemas de decisión

A pesar de que informalmente se usan las clases P , NP y NP -Hard, para clasificar prácticamente cualquier problema computacional, estas categorías aplican formalmente solo a **problemas de decisión**, es decir, problemas que se pueden responder con un sí o un no.

De esos problemas, no todos se pueden resolver con un computador, incluso cuando se pueden modelar matemáticamente. Los círculos externo de la figura 1 representan únicamente los problemas de decisión que son computables, que se llaman **decidibles**.

Muchos de los problemas de algoritmia son problemas de optimización, en donde

Parte IV

Apéndices

Apéndice A. Sumatorias

Apéndice B. Iteración

La iteración es una forma de expresar repetición.

Apéndice C. Ecuaciones recursivas

3.1. Relación con ecuaciones diferenciales

Quando yo estudié este tema por primera vez, sentía que estaba volviendo a estudiar ecuaciones diferenciales. Da la impresión de que son isomórficas, pero no lo son. Añado esta sección para ahondar en eso.

Las ecuaciones de recurrencia son, en muchos aspectos, el análogo discreto de las ecuaciones diferenciales

Apéndice D. Multiplicación matricial

La multiplicación de matrices es una generalización del producto punto entre vectores. Si el número de columnas de $\mathbf{A}$ es igual al número de filas de $\mathbf{B}$, el producto $\mathbf{AB}$ está definido y se dice que $\mathbf{A}$ y $\mathbf{B}$ son compatibles bajo la multiplicación.

El producto de dos matrices $\mathbf{A}_{p \times q}$ y $\mathbf{B}_{q \times r}$ consiste en el producto punto de cada fila de $\mathbf{A}$ con cada columna de $\mathbf{B}$. En la matriz resultado $\mathbf{C}_{p \times r}$, el elemento c_{ij} que está la i -ésima fila y j -ésima columna de $\mathbf{C}$, se computa como el producto punto de la i -ésima fila de $\mathbf{A}$ y la j -ésima columna de $\mathbf{B}$:

$$c_{ij} = \sum_{k=1}^q a_{ik} b_{kj}$$

Es decir: se multiplica el primer elemento en la i -ésima fila de $\mathbf{A}$ por el primer elemento de la j -ésima columna de $\mathbf{B}$, luego el segundo de cada una, el tercero, y así; y luego su suma da lugar al elemento c_{ij} .

Es por eso que el número de columnas de $\mathbf{A}_{p \times q}$ debe ser igual al número de filas de $\mathbf{B}_{q \times r}$: para que las p filas de $\mathbf{A}$ (cada una con q elementos, la cantidad de columnas) tengan cada una la misma cantidad de elementos que cada columna de $\mathbf{B}$ (cada una con q elementos, la cantidad de filas) y el producto punto esté bien definido. $\mathbf{C}$ es de tamaño $p \times r$ porque hereda la cantidad de filas de $\mathbf{A}$ y el número de columnas de $\mathbf{B}$.

Se hace evidente que la multiplicación de matrices no es conmutativa: en general, $\mathbf{AB} \neq \mathbf{BA}$, pues si las matrices no son cuadradas los dos productos no tienen el mismo tamaño, muchas veces ni siquiera están ambos definidos. Esto hace que sea importante aclarar si se multiplica “por derecha” o “por izquierda”.

Nótese que la multiplicación de una matriz $\mathbf{A}_{m \times n}$ por un vector $\mathbf{v}$ solo está definida por derecha si $\mathbf{v} \in \mathbb{R}^n$ (y es, por convención, un vector columna), pues el vector toma el rol de una matriz $n \times 1$. Similarmente, esa multiplicación solo está definida por izquierda si $\mathbf{v} \in \mathbb{R}^m$ y se usa $\mathbf{v}^T$ (o sea la transpuesta: se usa como vector fila o como matriz $1 \times n$). La multiplicación entre una matriz y un vector columna resulta en un vector columna, como el producto entre una matriz y un vector fila es un vector fila.

La multiplicación de matrices es asociativa, las multiplicaciones en cadena se pueden agrupar en cualquier orden sin alterar la secuencia de las matrices:

$$\mathbf{ABC} = (\mathbf{AB})\mathbf{C} = \mathbf{A}(\mathbf{BC})$$

Esto se debe a que cada elemento del resultado final se computa mediante la sumatoria de productos entre escalares que se mostró recién. Al encadenar multiplicaciones, lo que se genera es una secuencia de sumatorias anidadas. Como la suma y la multiplicación de números reales son conmutativas y asociativas, el orden en que se resuelvan estas sumatorias no altera el resultado.

Apéndice E. Números de Catalán